LeFusion: Architecture and How We Use It

Inserting aneurysms into real CTA patches

Prepared for Charitha · November 2026

In one line

LeFusion is a 3D diffusion model that learns to draw *only the lesion* inside a mask. When generating, everything outside the mask is taken from a real scan, so the background stays real and the mask is exact.

Paper: H. Zhang *et al.*, "LeFusion: Controllable Pathology Synthesis via Lesion-Focused Diffusion Models", ICLR 2025 [1]. **Code** (Apache-2.0): `https://github.com/HINTLab/LeFusion`. All details below were checked against the released code. Where the paper says something different, Section 5 lists it.

Contents

1 What LeFusion does **2**

2 The network **2**

2.1 3D U-Net . . . 2

2.2 How the time step and histogram enter . . . 3

3 Training **4**

3.1 Inputs, output, loss . . . 4

3.2 Settings in the code . . . 4

4 Inference (generation) **4**

4.1 Inputs and output . . . 4

4.2 What happens at each step . . . 5

4.3 Histogram types . . . 5

5 Paper vs. code **5**

6 How we use it for aneurysm CTA **6**

6.1 Lung (paper) vs. our data . . . 6

6.2 Folders and file names (required by the code) . . . 6

6.3 Code changes . . . 6

6.4 Training flow . . . 7

6.5 Generation flow . . . 7

6.6 Commands . . . 8

6.7 Checks . . . 8

1 What LeFusion does

- **Training:** take real lesion patches, add noise, and teach the network to predict that noise. The error is counted *only inside the lesion mask*.
- **Generation:** take a *normal* patch, a mask and a histogram. Start from noise. At every step, the network draws the lesion, and the area outside the mask is replaced by the real normal patch.

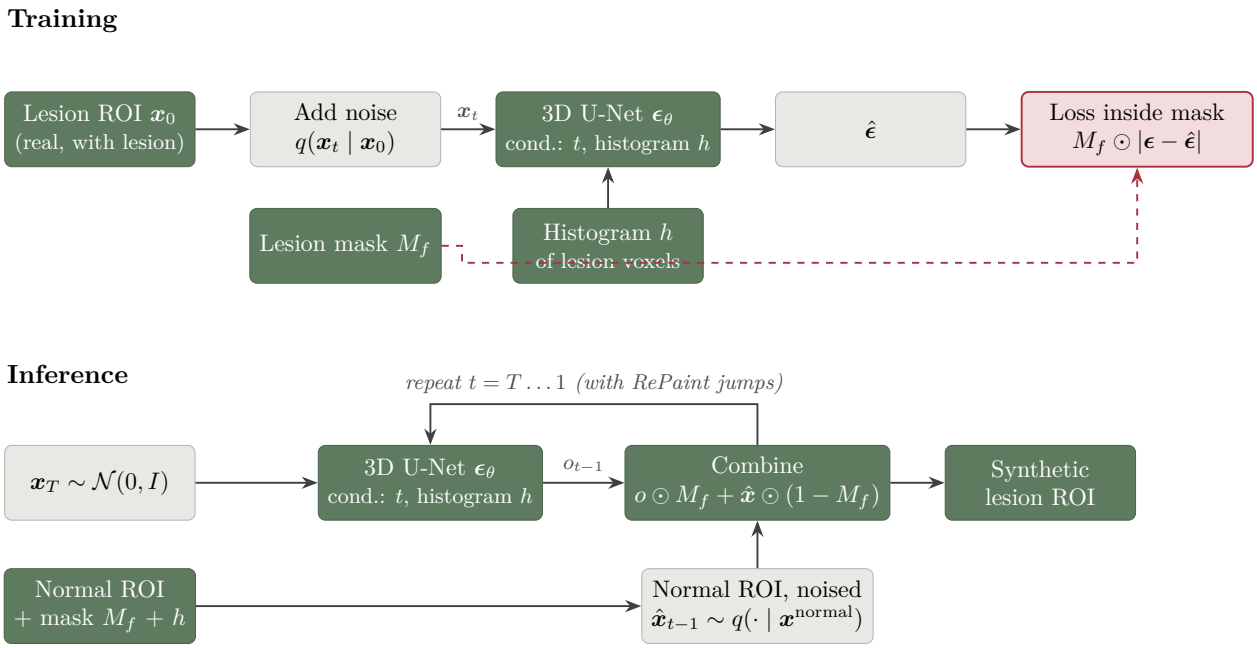


Figure 1: LeFusion overview (redrawn from the paper's framework figure). Top: training uses only real lesion ROIs and a lesion-focused loss. Bottom: inference keeps the real normal background and generates only inside M_f.

2 The network

2.1 3D U-Net

The network is a "video-style" 3D U-Net. It treats depth like time in a video: it works inside each slice and across slices separately.

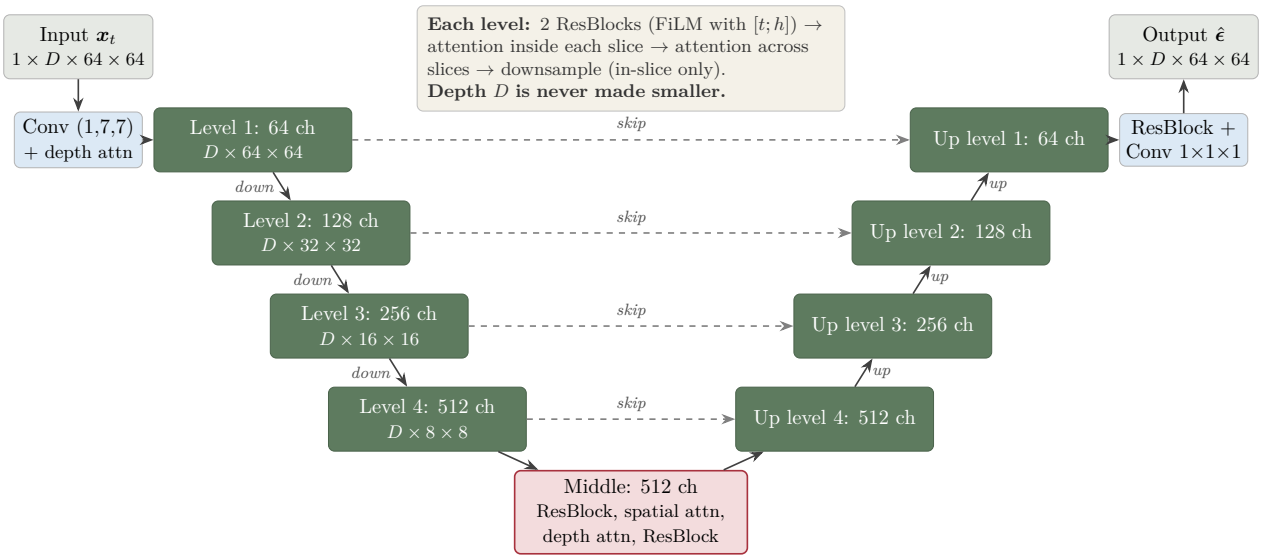


Figure 2: Denoising U-Net of LeFusion [code]: base width 64, channel multipliers $(1, 2, 4, 8)$, four levels. Down- and up-sampling act only on the in-slice size; the depth stays the same through the whole network.

Part	What it does
First layer	conv 1×7×7 (inside each slice), then attention across slices
Channels	64, 128, 256, 512 (base = image size 64, multipliers 1, 2, 4, 8)
ResBlock	two conv blocks (kernel 1×3×3, GroupNorm, SiLU); the *first* block gets the FiLM condition
Attention inside a slice	fast (linear) attention, 8 heads
Attention across slices	normal attention along depth, with position encoding
Middle	full attention inside slices + attention across slices
Down / up	only the in-slice size changes; depth stays the same
Output	predicted noise $\hat{\boldsymbol{\epsilon}}$, same size as the input

2.2 How the time step and histogram enter

1. The time step t becomes a 256-number vector $\boldsymbol{e}_t$ (sinusoidal code + small MLP).
2. The histogram h (16 numbers) is joined to it: $\boldsymbol{c} = [\boldsymbol{e}_t\,; h]$ (272 numbers).
3. In each ResBlock, $\boldsymbol{c}$ gives a scale γ and shift β per channel (FiLM):

$$[\gamma, \beta] = \mathrm{Linear}(\mathrm{SiLU}(\boldsymbol{c})), \qquad \boldsymbol{f}' = \mathrm{GroupNorm}(\boldsymbol{f}) \cdot (1 + \gamma) + \beta. \tag{1}$$

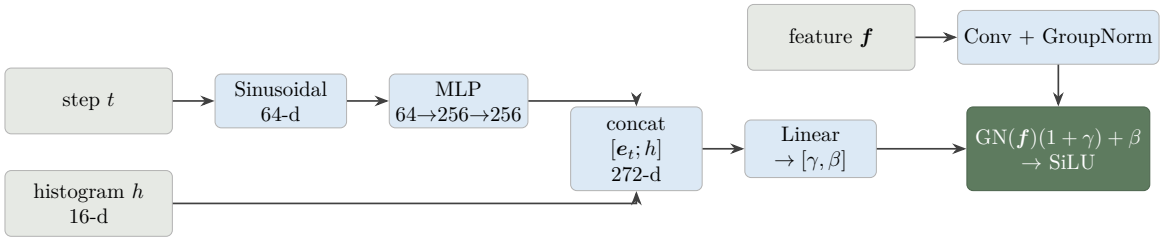


Figure 3: How t and the histogram h condition each ResBlock (FiLM) [code].

Noise schedule: cosine, $T = 300$ steps:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\!\Big(\frac{t/T + 0.008}{1.008} \cdot \frac{\pi}{2}\Big). \tag{2}$$

3 Training

3.1 Inputs, output, loss

Inputs to the network	noisy patch $\boldsymbol{x}_t$ (1 channel), time step t, histogram h (16 numbers)
Not an input	the mask M_f – it is used only in the loss
Output	predicted noise $\hat{\boldsymbol{\epsilon}}$ (same size as the patch)
Loss	L1 error between real and predicted noise, inside the mask only

Add noise:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}). \tag{3}$$

Loss (code):

$$\mathcal{L} = \frac{1}{|\Omega|} \sum_{v \in \Omega} \left| M_f(v)\,\epsilon(v) - M_f(v)\,\hat{\epsilon}(v) \right|, \tag{4}$$

where Ω = all voxels in the batch. Because it divides by *all* voxels, small lesions give small loss values.

Histogram (code): 16 bins over the whole normalised range $[-1, 1]$, divided by the number of mask voxels:

```
hist = torch.histc(img[mask > 0], bins=16, min=-1, max=1) / mask.sum()
```

3.2 Settings in the code

Setting	Value
Patch size	$32 \times 64 \times 64$ (depth first), hard-coded in the dataset file
Intensity	HU clipped to $[-1000, 400]$ (lung), scaled to $[-1, 1]$, hard-coded
Augmentation	random flip on one axis
Steps T	300, cosine
Optimiser	Adam, learning rate 10^{-4}
Batch	20, gradient accumulation 2
Training steps	50,001 (README: 20k–50k is enough)
EMA	0.995, every 10 steps, from step 2,000
Checkpoints	every 1,000 steps: `model-N.pt` (keys `model` and `ema`)
Compute (README)	5×A100 40 GB, 2.5 days for 50k steps

Algorithm 1 One training step

1: Take a lesion patch $\boldsymbol{x}_0$, its mask M_f, its histogram h
2: Pick a random step t and random noise $\boldsymbol{\epsilon}$
3: $\boldsymbol{x}_t \leftarrow \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ ▷ whole patch is noised
4: $\hat{\boldsymbol{\epsilon}} \leftarrow \text{U-Net}(\boldsymbol{x}_t, t, h)$
5: $\mathcal{L} \leftarrow \text{mean}\,|M_f \odot \boldsymbol{\epsilon} - M_f \odot \hat{\boldsymbol{\epsilon}}|$
6: Update weights; update EMA copy

4 Inference (generation)

4.1 Inputs and output

Inputs	normal patch $\boldsymbol{x}^{\text{normal}}$ (no lesion), mask M_f (where to draw), histogram h (how it looks)
Output	synthetic patch with a lesion inside M_f; the mask M_f is saved with it

4.2 What happens at each step

At every step, *first* the outside of the mask is replaced by the real patch (with the right noise), *then* the network takes one denoising step:

$$\hat{\boldsymbol{x}}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}^{\text{normal}} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon} \qquad \text{(real patch, noised)} \tag{5}$$

$$\boldsymbol{x}_t \leftarrow \hat{\boldsymbol{x}}_t \odot (1 - M_f) + \boldsymbol{x}_t \odot M_f \qquad \text{(combine)} \tag{6}$$

$$\boldsymbol{x}_{t-1} = \text{DDPM step from U-Net}(\boldsymbol{x}_t, t, h) \qquad \text{(denoise whole patch)} \tag{7}$$

Going back and forth (RePaint). The code sometimes adds noise back, $\boldsymbol{x}_t = \sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{n}$, and denoises again, so the lesion blends with its surroundings [2]. Settings: `jump_length` and `jump_n_sample` (script: 5 and 5; README: 2–10 works; about 40 s per patch on an A100 with 2 and 2).

Algorithm 2 Generation (simplified)

Inputs: normal patch $\boldsymbol{x}^{\text{normal}}$, mask M_f, histogram h
$\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
for each step in the RePaint schedule **do**
  if going down **then**
    replace outside of M_f with $\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}^{\text{normal}} + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$
    $\boldsymbol{x}_{t-1} \leftarrow$ one denoising step with U-Net$(\boldsymbol{x}_t, t, h)$
  else add one step of noise back
  end if
end for
return $\boldsymbol{x}_0$ and M_f

Watch out

The last step is a network step, so the final output is from the network *everywhere*, also outside the mask (very close to the real patch, but not identical). **After generation, put the real background back outside M_f.**

4.3 Histogram types

The code loads 3 histograms (cluster centres) from `hist_clusters/lidc_clusters.json` (3×16 numbers). For lung nodules they are roughly ground-glass, part-solid and solid. Each is changed by up to $\pm 10\%$ at random. With `types=3`, every patch is generated 3 times, once per histogram, into folders `Image_1..3`, `Mask_1..3`.

Weights: inference loads the `model` weights, not the `ema` weights.

5 Paper vs. code

Topic	Paper says	Code does
Histogram	cross-attention	joined to time step, FiLM in the first conv of each ResBlock
Loss	L_2 inside mask	L1 inside mask, averaged over all voxels
Steps	not given	300, cosine
Combine step	after the model step	before each model step; final output is all network
Compute	≈3 A100, 1 day	README: 5 A100, 2.5 days

6 How we use it for aneurysm CTA

6.1 Lung (paper) vs. our data

	Paper (lung)	**Ours (aneurysm CTA)**
Patch	$32 \times 64 \times 64$ around a nodule	64^3 at 0.45 mm around an aneurysm (small random shift)
Intensity	$[-1000, 400]$ HU	$[-100, 900]$ HU
Number of patches	2,104	≈1,300 (full data); ≈117 now (pilot only)
Histogram clusters	lung clusters	make our own 3 clusters from our training histograms
Normal patches	healthy lungs	vessel branches >15 mm from any aneurysm
Masks at generation	real or DiffMask	shape library (real shapes, resized, rotated, placed on the vessel wall)

6.2 Folders and file names (required by the code)

```
data/AneurysmCTA/
  Pathological/                      <- training
    Image/  Tr0001_Vol_01.nii.gz      (aneurysm patch)
    Mask/   Tr0001_Mask_01.nii.gz     ("Vol_" -> "Mask_")
    test.txt                          (names to EXCLUDE, no .nii.gz)
  Normal/                            <- generation
    Image/  Tr0123_CVol_01.nii.gz     (normal vessel patch; must contain "_CVol_")
    Mask/   Tr0123_CMask_01.nii.gz    (placed mask)
```

- The mask is found by replacing `Vol_` with `Mask_` in the file name.
- Generation splits the name at `_CVol_`; without it, saving crashes.
- `test.txt` lists validation patches that are left out (it is also applied during generation).

6.3 Code changes

1. Intensity window and patch size – in *both* `LeFusion/dataset/lidc_hist.py` and `lidc_hist_in.py`:

```
tio.Clamp(out_min=-100, out_max=900)
tio.RescaleIntensity(in_min_max=(-100, 900), out_min_max=(-1.0, 1.0))
tio.CropOrPad(target_shape=(64, 64, 64))     # image AND mask transforms
```

2. Depth – set `diffusion_depth_size=64` in the train and inference scripts. Depth is never made smaller in the U-Net, so memory is about 2× the paper's: use batch 4–8.

3. Histogram clusters – make `hist_clusters/lidc_clusters.json` from our data: run k-means ($k = 3$) on our training histograms (low / medium / high contrast). Same format: `[{"n_class": 3, "centers": [[16 numbers], ...]}]`.

4. Smaller fixes

- `lidc_train.sh`: `results_folder` points to `EMIDEC`; set your own folder.
- `num_workers` is 20 by default; use 2–4 on Colab or Kaggle.
- Install from the repo root (`pip install -r requirements.txt`); the README's `LeFusion_LIDC` folder does not exist.

- Optional: load `ema` weights at inference; average the loss over mask voxels only; give the mask as a 2nd input channel (bigger change).

6.4 Training flow

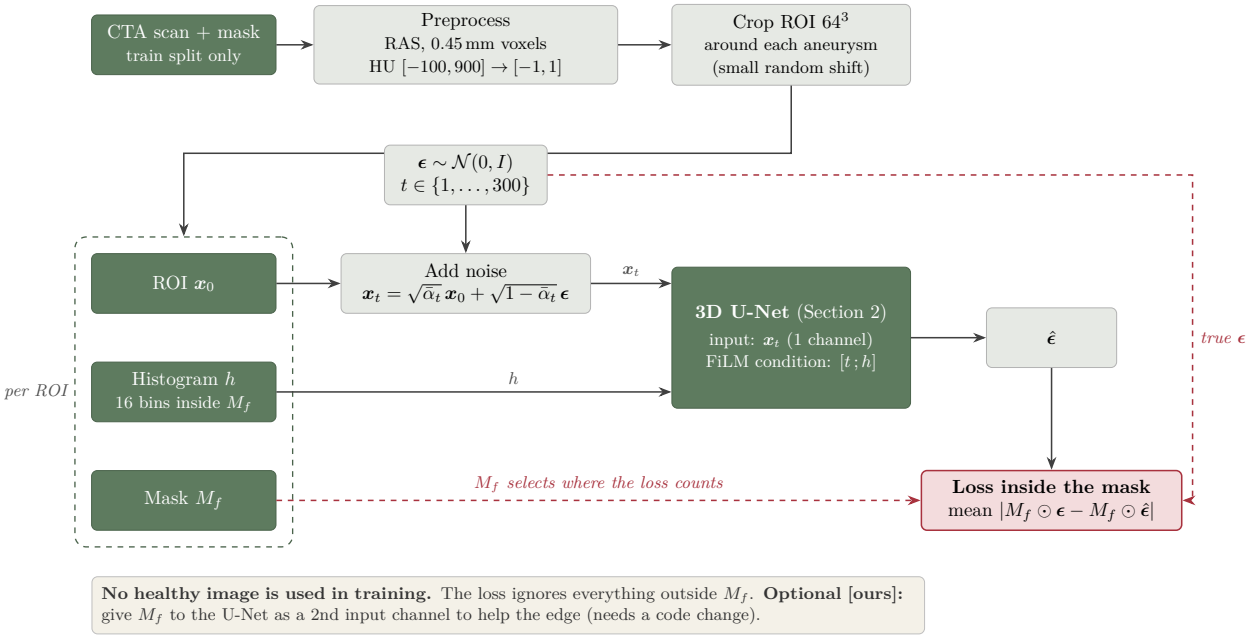


Figure 4: Our training flow. Only real aneurysm ROIs, their masks and their histograms are used.

6.5 Generation flow

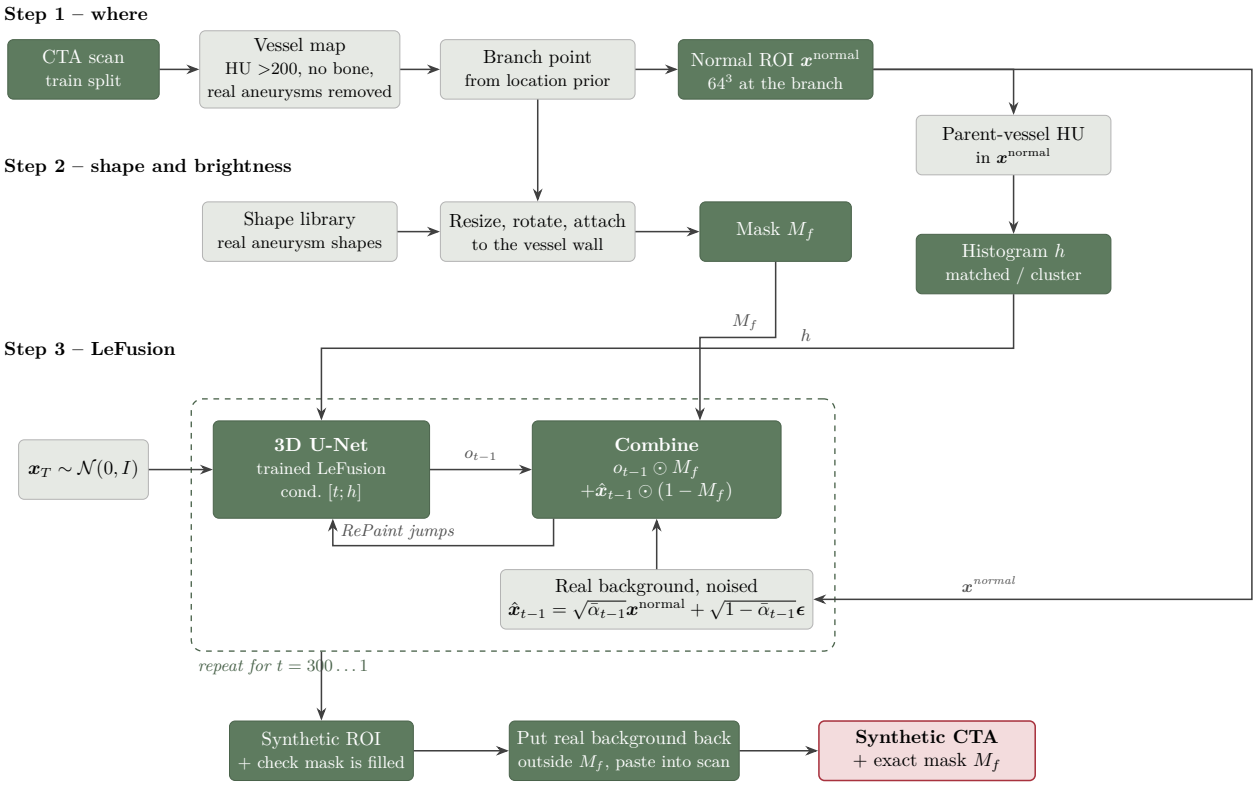


Figure 5: Our inference flow without DiffMask. The mask comes from the shape library; the histogram is matched to the parent vessel or taken from one of our 3 clusters.

6.6 Commands

Train:

```
python LeFusion/train/train.py dataset=lidc \
  model.diffusion_img_size=64 model.diffusion_depth_size=64 \
  model.diffusion_num_channels=1 model.cond_dim=16 \
  model.batch_size=6 model.train_num_steps=50001 \
  dataset.root_dir=data/AneurysmCTA/Pathological/Image \
  dataset.test_txt_dir=data/AneurysmCTA/Pathological/test.txt \
  model.results_folder=LeFusion/LeFusion_Model/CTA
```

Generate:

```
python LeFusion/inference/inference.py data_type=lidc types=3 \
  diffusion_img_size=64 diffusion_depth_size=64 diffusion_num_channels=1 \
  batch_size=3 cond_dim=16 \
  test_txt_dir=data/AneurysmCTA/Pathological/test.txt \
  dataset_root_dir=data/AneurysmCTA/Normal/Image/ \
  target_img_path=data/AneurysmCTA/gen/Image/ \
  target_label_path=data/AneurysmCTA/gen/Mask/ \
  model_path=LeFusion/LeFusion_Model/CTA/model-50.pt \
  schedule_jump_params.jump_length=5 schedule_jump_params.jump_n_sample=5
```

(`dataset=lidc` / `data_type=lidc` just chooses the LIDC code path, which we edited.)

6.7 Checks

- **Is the mask filled?** Inside M_f, at least 70% of voxels should be bright (HU $>$ 200). If not, reject the sample.
- **Look** at 30–50 samples: the aneurysm should grow from the vessel, with no seam.
- **Brightness:** compare HU inside synthetic vs. real aneurysms.
- **No copying:** no sample should be almost the same as a training patch.
- **Real test:** nnU-Net trained on real vs. real + synthetic, tested on real scans only.

Watch out

With 99 scans (≈117 aneurysms) the model will likely copy training examples – use it only to test the pipeline. Leaving the lung window in the code is the easiest mistake to miss.

References

[1] H. Zhang, Y. Liu, J. Yang, S. Wan, X. Wang, W. Peng, P. Fua. LeFusion: Controllable pathology synthesis via lesion-focused diffusion models. *ICLR*, 2025. arXiv:2403.14066.

[2] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, L. Van Gool. RePaint: Inpainting using denoising diffusion probabilistic models. *CVPR*, 2022.